

Sistemas Expertos – Parte UNO

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá



12 de febrero de 2026

Definición y características.

- ▶ Un **sistema experto** es un programa de computadora que simula la capacidad de toma de decisiones de un experto humano.
- ▶ Características principales:
 - ▶ Base de conocimientos (hechos y reglas).
 - ▶ Motor de inferencia para razonar con la información.
 - ▶ Capacidad para explicar sus conclusiones.
 - ▶ Interfaz para interacción con el usuario.

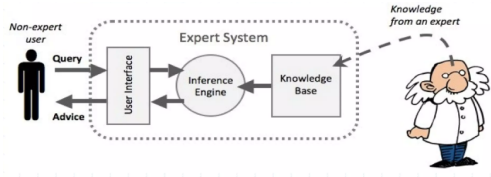


Imagen tomada de: [Internet](#).

Historia y evolución.

- ▶ Surgieron en la década de 1960 como parte de la inteligencia artificial.
- ▶ Primeros ejemplos: *DENDRAL* (diagnóstico químico), *MYCIN* (diagnóstico médico).
- ▶ Evolución hacia sistemas más complejos y aplicaciones en diversas áreas.

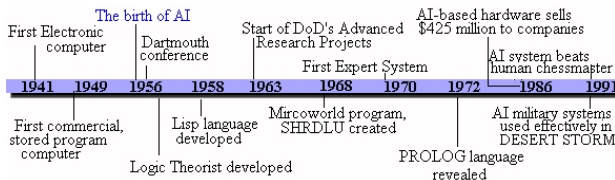


Imagen tomada de: [Internet](#).

Diferencias con sistemas tradicionales.

- ▶ Los sistemas tradicionales siguen un **conjunto fijo de instrucciones**.
- ▶ Los sistemas expertos manejan **incertidumbre y conocimiento impreciso**.
- ▶ Pueden aprender y adaptarse a nueva información.
- ▶ Ejemplo: Un sistema tradicional para cálculo contable vs. un sistema experto para asesoría fiscal.

Aplicaciones y ejemplos reales.

- ▶ **Medicina:** diagnóstico de enfermedades infecciosas.
 - ▶ Ejemplo: El sistema *MYCIN* recomendaba antibióticos con base en síntomas y análisis de laboratorio.
- ▶ **Ingeniería:** mantenimiento predictivo en maquinaria industrial.
 - ▶ Ejemplo: Un sistema experto analiza vibraciones y temperatura para predecir fallos en motores eléctricos.
- ▶ **Finanzas:** evaluación de solicitudes de crédito.
 - ▶ Ejemplo: Un sistema experto evalúa historial crediticio y nivel de ingresos para recomendar aprobación o rechazo de préstamos.
- ▶ **Agricultura:** planificación del riego y control de plagas.
 - ▶ Ejemplo: Un sistema experto considera humedad del suelo y previsión climática para decidir cuándo regar.

Participantes en el desarrollo de sistemas expertos.

El desarrollo de un sistema experto involucra distintos tipos de especialistas, cada uno con un rol esencial:

- ▶ **Experto del dominio (Domain Expert).**
- ▶ **Ingeniero del conocimiento (Knowledge Engineer).**
- ▶ **Usuario final (End User).**

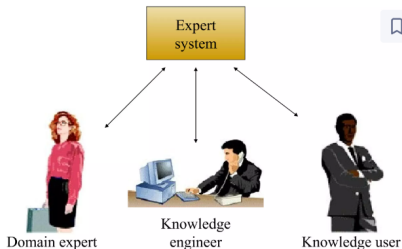


Imagen tomada de: [Internet.](#)

Experto del dominio (Domain Expert):

- ▶ Posee el **conocimiento especializado** en el área (medicina, ingeniería, etc.).
- ▶ Su conocimiento es la **fuentes para construir la base de conocimientos**.
- ▶ Ejemplo: un médico que describe cómo diagnosticar una infección.

Ingeniero del conocimiento (Knowledge Engineer):

- ▶ Traduce el conocimiento del experto del dominio en **reglas y estructuras formales**.
- ▶ **Construye la base de conocimientos y el motor de inferencia**.
- ▶ Ejemplo: diseña las reglas lógicas basadas en los síntomas médicos.

Usuario final (End User):

- ▶ **Utiliza el sistema** para resolver problemas o tomar decisiones.
- ▶ Debe poder **interactuar fácilmente** con el sistema mediante la **interfaz de usuario**.
- ▶ Ejemplo: un médico general que usa el sistema para recibir sugerencias diagnósticas.

Componentes de un Sistema Experto.

Un sistema experto está compuesto por varios módulos que interactúan entre sí:

- ▶ Base de conocimientos
- ▶ Motor de inferencia
- ▶ Interfaz de usuario
- ▶ Módulo de explicación

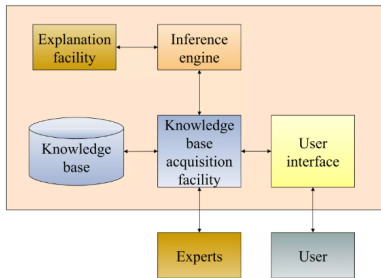


Imagen tomada de: [Internet](#).

Cada uno cumple un rol clave en la toma de decisiones automatizada y justificada.

Base de conocimientos.

- ▶ Contiene el conocimiento del dominio en forma de:
 - ▶ **Hechos:** información concreta y verificable.
 - ▶ **Reglas:** representadas como producciones del tipo:

SI condición ENTONCES acción

- ▶ Ejemplo:

SI temperatura $> 38^{\circ}C$ $\wedge$ dolor de cabeza $\Rightarrow$ posible fiebre

- ▶ Puede mantenerse por expertos humanos o aprenderse automáticamente.

Motor de inferencia.

- ▶ Es el “**razonador**” del sistema: **aplica reglas sobre los hechos conocidos para obtener conclusiones.**
- ▶ Emplea estrategias de inferencia:
 - ▶ **Encadenamiento hacia adelante (forward chaining):** desde hechos hacia conclusiones.
 - ▶ **Encadenamiento hacia atrás (backward chaining):** desde hipótesis hacia hechos necesarios.

Ejemplo común

- ▶ Hechos conocidos:

$$A = \text{“fiebre”}, \quad B = \text{“tos”}$$

- ▶ Regla:

$$A \wedge B \Rightarrow C$$

donde $C = \text{“infección respiratoria”}$

Encadenamiento hacia adelante (Forward Chaining)

- ▶ Parte de los hechos disponibles (A y B).
- ▶ Evalúa qué reglas pueden activarse.
- ▶ Como se cumple $A \wedge B$, se infiere C .
- ▶ Dirección del razonamiento:

Hechos $\rightarrow$ Reglas $\rightarrow$ Conclusión

Encadenamiento hacia atrás (Backward Chaining)

- ▶ Parte de una hipótesis: ¿es verdadera C ?
- ▶ Busca reglas que concluyan C .
- ▶ Encuentra $A \wedge B \Rightarrow C$.
- ▶ Verifica si A y B son verdaderos en la base de hechos.
- ▶ Dirección del razonamiento:

Conclusión $\rightarrow$ Reglas $\rightarrow$ Hechos

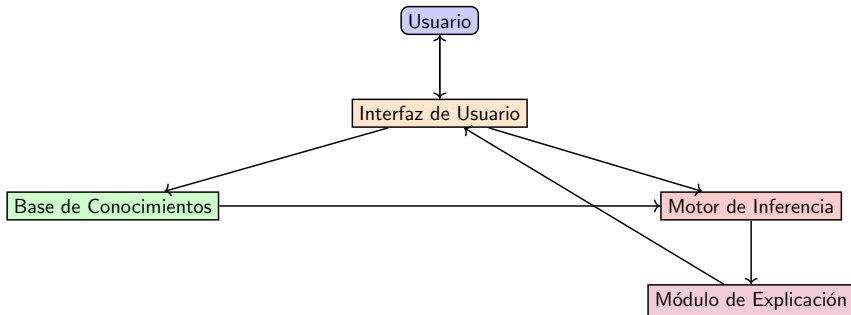
Interfaz de usuario.

- ▶ **Permite la interacción entre el usuario y el sistema experto.**
- ▶ Funciones:
 - ▶ Recibir datos del usuario (entrada de hechos).
 - ▶ Presentar resultados o recomendaciones.
 - ▶ Preguntar al usuario cuando faltan datos.
- ▶ Ejemplo: Un sistema médico que solicita síntomas y recomienda un tratamiento basado en el análisis.

Módulo de explicación y justificación.

- ▶ Permite al sistema justificar sus conclusiones ante el usuario.
- ▶ Muestra:
 - ▶ Qué reglas se usaron.
 - ▶ Qué hechos activaron cada regla.
- ▶ Ejemplo:
 - ▶ “Se concluyó que hay infección respiratoria porque se detectó fiebre y tos, y se aplicó la regla R3.”
- ▶ Mejora la confianza del usuario en el sistema.

Módulo de explicación y justificación.



Ejemplo en Python: Motor de Inferencia Simple.

Objetivo

Aplicar reglas básicas del tipo: “SI fiebre y dolor de cabeza y congestión, ENTONCES Puede tener GRIPE”.

► **Ver Notebook.**

¿Qué es la Representación del Conocimiento?

- ▶ Es una forma de modelar información que un sistema puede procesar y razonar.
- ▶ Objetivo: facilitar la toma de decisiones, inferencias y aprendizaje.
- ▶ Criterios: expresividad, eficiencia computacional, comprensibilidad y robustez.

Formas de Representar Conocimiento.

- ▶ Reglas de producción (if-then)
- ▶ Marcos (frames)
- ▶ Redes semánticas
- ▶ Lógica proposicional y lógica de primer orden

Reglas de Producción.

- ▶ Las reglas de producción siguen la estructura:
SI (condición) ENTONCES (acción).
- ▶ Son la base de muchos sistemas expertos basados en reglas.
- ▶ Se almacenan en la **base de conocimientos**.
- ▶ El **motor de inferencia** las evalúa continuamente.

Ejemplo:

- ▶ SI temperatura > 38°C Y tos = sí ENTONCES
diagnóstico = infección respiratoria
- ▶ Se **pueden combinar muchas reglas** para realizar inferencias complejas.
- ▶ Es compatible con estrategias de encadenamiento hacia adelante y hacia atrás.

Regla	Condición (SI)	Acción (ENTONCES)
R1	temperatura $\geq 38^{\circ}\text{C}$ Y tos = sí	diagnóstico = infección respiratoria
R2	dolor de garganta Y fiebre	diagnóstico = faringitis
R3	contacto con alérgenos Y estornudos frecuentes	diagnóstico = alergia
R4	presión arterial $> 140/90$	diagnóstico = hipertensión

- ▶ Cada regla puede evaluarse por separado.
- ▶ **Se activa cuando se cumplen todas las condiciones de la izquierda.**
- ▶ **El SE puede aplicar múltiples reglas según los hechos disponibles.**

Marcos (Frames).

- ▶ Un **frame** representa un concepto, objeto o situación.
- ▶ Está compuesto por **slots** (ranuras), que contienen:
 - ▶ **Atributos** (nombre, tipo, valor, etc.).
 - ▶ **Reglas asociadas** (condiciones y acciones).
 - ▶ **Relaciones con otros marcos** (herencia, agregación).
- ▶ Permiten organizar el conocimiento jerárquicamente y reutilizar estructuras.

Ejemplo: Un frame que representa un “Animal”

- ▶ Slots: nombre, tipo, número de patas, puede volar?

Frame	ISA	Tipo	Patas	¿Vuela?
Animal	–	genérico	–	–
Pájaro	Animal	Ave	2	Sí
Pingüino	Pájaro	Ave	2	No

- ▶ **ISA:** relación de herencia entre frames.
- ▶ Los valores se heredan y pueden ser sobrescritos.
- ▶ Ejemplo: el Pingüino hereda de Pájaro, pero cambia el valor de “¿Vuela?”.

Ejemplo en Python: Marcos (frames).

Objetivo

Simular un sistema experto basado en marcos, destacando que:

- ▶ El conocimiento se organiza en clases jerárquicas.
- ▶ Las propiedades (slots) se heredan automáticamente desde clases generales a más específicas.
- ▶ La lógica de inferencia está implícita en la estructura del conocimiento, sin necesidad de un motor de reglas aparte.

▶ **Ver Notebook.**

Redes Semánticas.

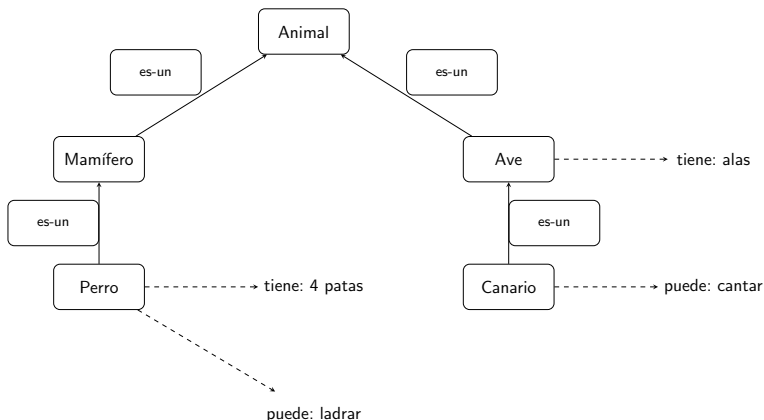
- ▶ Representan conocimiento como una red de nodos y relaciones.
- ▶ Cada **nodo** es un concepto (ej. “Perro”, “Animal”).
- ▶ Los **arcos** indican relaciones semánticas:
 - ▶ es-un: **relación jerárquica**.
 - ▶ tiene: **relación de atributo**.
 - ▶ puede: **capacidad o función**.
- ▶ Permiten inferencias por propagación de propiedades.

Representación del Conocimiento en Sistemas Expertos

Nodo origen	Relación	Nodo destino
Perro	es-un	Mamífero
Mamífero	es-un	Animal
Perro	tiene	4 patas
Perro	puede	ladrar
Ave	tiene	alas
Canario	es-un	Ave
Canario	puede	cantar

- ▶ El concepto “Canario” hereda que “tiene alas” por ser un “Ave”.
- ▶ Se permite inferencia de propiedades mediante encadenamiento.

Representación del Conocimiento en Sistemas Expertos



- ▶ Las flechas continuas indican herencia (**es-un**),
- ▶ las discontinuas indican propiedades.

Ejemplo en Python: Redes semánticas.

Objetivo

Mostrar cómo representar conocimiento mediante una red semántica con nodos y relaciones jerárquicas, incluyendo propiedades, y realizar razonamiento básico por búsqueda recursiva para clasificar conceptos.

► **Ver Notebook.**

Lógica Proposicional.

- ▶ Representa el conocimiento con proposiciones lógicas.
- ▶ Cada proposición puede ser verdadera o falsa.
- ▶ Se combinan usando conectores:
 - ▶ $\neg P$ (no P), $P \wedge Q$, $P \vee Q$, $P \rightarrow Q$, $P \leftrightarrow Q$
- ▶ Permite deducir conocimiento aplicando reglas lógicas.

Ejemplo:

- ▶ P : "Está lloviendo"
- ▶ Q : "La calle está mojada"
- ▶ $P \rightarrow Q$: "Si llueve, entonces la calle está mojada"

Ejemplo en lógica proposicional: reglas y hechos

Símbolo	Interpretación
P	El paciente tiene fiebre
Q	El paciente tiene infección
R	El paciente debe tomar antibiótico
$P \rightarrow Q$	Si el paciente tiene fiebre, entonces tiene una infección
$Q \rightarrow R$	Si el paciente tiene una infección, debe tomar antibiótico

- **Deducción:** si P es verdadero, entonces Q es verdadero, y por lo tanto R también.

Conectores lógicos en lógica proposicional

Símbolo	Nombre	Significado
$\neg P$	Negación	"No P" – Invierte el valor de verdad de P
$P \wedge Q$	Conjunción	"P y Q" – Verdadero solo si ambos son verdaderos
$P \vee Q$	Disyunción inclusiva	"P o Q" – Verdadero si al menos uno es verdadero
$P \oplus Q$	Disyunción exclusiva (XOR)	"P o Q pero no ambos" – Verdadero si solo uno es verdadero
$P \rightarrow Q$	Implicación	"Si P, entonces Q" – Falsa solo si P es verdadero y Q falso
$P \leftrightarrow Q$	Bicondicional	"P si y solo si Q" – Verdadero si ambos tienen el mismo valor

- ▶ Estos conectores permiten construir reglas complejas a partir de hechos simples.
- ▶ Son la base de la inferencia en lógica formal y sistemas expertos basados en lógica.

Ejemplo en Python: Lógica proposicional.

Objetivo

Representar conocimiento mediante proposiciones booleanas y aplicar reglas if-then para inferir conclusiones de forma lógica y determinista.

► **Ver Notebook.**

Lógica de Primer Orden.

- ▶ Extiende la lógica proposicional con:
 - ▶ **Predicados:** describen propiedades o relaciones (ej. $\text{Tiene}(x, \text{fiebre})$)
 - ▶ **Cuantificadores:** $\forall$ (para todo), $\exists$ (existe)
 - ▶ **Variables:** representan elementos del dominio
- ▶ **Ejemplo:**
 - ▶ Hecho: $\text{Paciente}(\text{juan})$
 - ▶ Hecho: $\text{Tiene}(\text{juan}, \text{fiebre})$
 - ▶ Regla: $\forall x (\text{Tiene}(x, \text{fiebre}) \rightarrow \text{TieneInfección}(x))$
- ▶ **Inferencia:** dado que Juan tiene fiebre, se concluye que tiene infección.

Ejemplo con lógica de primer orden.

► Hechos:

Paciente(ana), Tiene(ana, fiebre), Tiene(ana, tos)

► Reglas:

$$\forall x \text{ (Tiene}(x, \text{fiebre}) \rightarrow \text{PuedeTenerInfección}(x))$$
$$\forall x \text{ (PuedeTenerInfección}(x) \wedge \text{Tiene}(x, \text{tos}) \rightarrow \text{Diagnóstico}(x, \text{bronquitis}))$$

► Inferencia:

Diagnóstico(ana, bronquitis)

► Interpretación: Ana tiene fiebre y tos, por lo tanto puede tener infección, y con tos se concluye diagnóstico de bronquitis.

Comparación entre Lógica Proposicional y Lógica de Primer Orden

Característica	Lógica Proposicional	Lógica de Primer Orden
Unidad básica	Proposiciones atómicas (verdadero/falso)	Predicados con variables y argumentos
Expresividad	Limitada, solo hechos simples y reglas directas	Muy expresiva, puede describir objetos, propiedades y relaciones complejas
Cuantificadores	No disponibles	Sí, cuantificadores universales ($\forall$) y existenciales ($\exists$)
Relaciones entre objetos	No pueden representarse directamente	Pueden representarse mediante predicados
Inferencia	Reglas básicas con conectores lógicos	Reglas con variables y cuantificadores, más flexible y poderosa
Ejemplo	$P \rightarrow Q$ (Si llueve, entonces la calle está mojada)	$\forall x (Llueve(x) \rightarrow Mojada(x))$
Complejidad computacional	Menor	Mayor, debido a variables y cuantificadores

Ejemplo en Python: Lógica de primer orden (lógica de predicados).

Objetivo

Modelar conocimiento con lógica de primer orden usando predicados e inferencia lógica sobre una base de hechos, permitiendo responder preguntas complejas mediante reglas formales.

► **Ver Notebook.**

Ontologías OWL.

- ▶ Las **ontologías OWL (Web Ontology Language)** permiten representar conocimiento de forma formal y lógica.
- ▶ Están diseñadas para la **Web Semántica**, permitiendo que las máquinas interpreten y razonen sobre los datos.

¿Qué es una ontología?

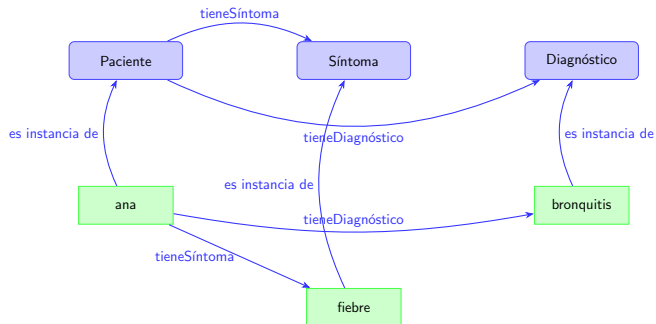
- ▶ Un modelo que:
 - ▶ Define **clases** (conceptos).
 - ▶ Establece **propiedades** (relaciones entre clases o individuos).
 - ▶ Describe **instancias** (individuos u objetos concretos).
 - ▶ Permite aplicar **reglas de inferencia lógica**.

Representación del Conocimiento en Sistemas Expertos

- Ejemplo: una ontología describe conceptos, relaciones e individuos con semántica formal.

Elemento	Nombre	Descripción
Clase	Paciente	Representa a una persona que recibe atención médica.
Clase	Síntoma	Concepto médico que describe un signo observado (ej. fiebre).
Clase	Diagnóstico	Resultado inferido a partir de los síntomas.
Propiedad	tieneSíntoma	Relaciona un paciente con un síntoma: tieneSíntoma(Paciente, Síntoma)
Propiedad	tieneDiagnóstico	Relaciona un paciente con un diagnóstico: tieneDiagnóstico(Paciente, Diagnóstico)
Individuo	ana:Paciente	Ana es una instancia de la clase Paciente.
Individuo	fiebre:Síntoma	Fiebre es un síntoma observado.
Individuo	bronquitis:Diagnóstico	Diagnóstico inferido a partir de los síntomas.

Diagrama visual de ontología OWL.



Ejemplo en Python: Ontología OWL.

Objetivo

Representar conocimiento estructurado y semántico mediante ontologías OWL en Python usando owlready2, incluyendo la definición de clases jerárquicas, propiedades, creación de instancias y almacenamiento en formato estándar, preparando la base para aplicar razonamiento automático y construir sistemas expertos semánticos.



Ver Notebook.

¿Qué es la Adquisición del Conocimiento?

- ▶ Proceso de extraer, estructurar y organizar el conocimiento desde fuentes humanas, documentales o computacionales.
- ▶ Es fundamental para construir la base de conocimientos de un sistema experto.
- ▶ Involucra la colaboración entre el experto en el dominio y el ingeniero del conocimiento.

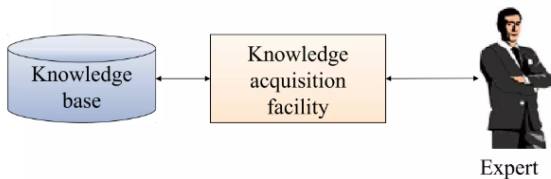


Imagen tomada de: [Internet](#).

Técnicas de Adquisición de Conocimiento.

- ▶ **Entrevistas estructuradas y no estructuradas**
- ▶ **Cuestionarios y encuestas**
- ▶ **Observación directa de tareas**
- ▶ **Análisis de protocolos (p. ej. pensar en voz alta)**
- ▶ **Extracción automática de datos** (text mining, machine learning)

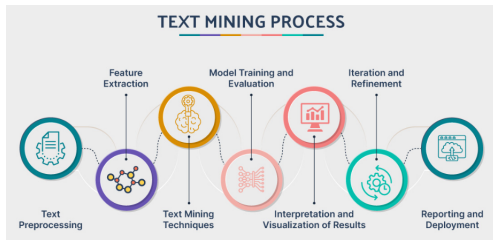


Imagen tomada de: [Internet](#).

Métodos de Adquisición.

▶ Manual:

- ▶ Conocimiento extraído directamente por expertos o ingenieros.
- ▶ Método **laborioso y detallado**, ideal para dominios específicos.

▶ Semi-automático:

- ▶ Apoyo de herramientas informáticas para facilitar la extracción.
- ▶ Combina **experiencia humana y procesamiento automatizado**.

▶ Automático:

- ▶ Uso de **algoritmos de aprendizaje para extraer reglas o relaciones**.
- ▶ Aplicado a grandes volúmenes de datos, por ejemplo minería de datos.

Rol del Experto y del Ingeniero.

► Experto del Dominio.

- Posee el **conocimiento** específico que se desea capturar.
- Participa **describiendo** sus procesos, decisiones y razonamientos.
- Proporciona **validación** del conocimiento adquirido.

► Ingeniero del Conocimiento.

- Actúa como **punto** entre el experto y el sistema.
- **Traduce** el conocimiento en estructuras lógicas o computacionales.
- Usa técnicas formales para representar el conocimiento.

Herramientas para Adquisición de Conocimiento.

- ▶ **Entornos de desarrollo de sistemas expertos** (p. ej. CLIPS, Jess)
- ▶ **Herramientas de minería de datos** (WEKA, Orange)
- ▶ **Sistemas de gestión del conocimiento** (Knowledge Management Systems)
- ▶ **Ontología y editores de conocimiento:** (Protégé)
 - ▶ Editor gratuito para construir ontologías OWL.
 - ▶ Permite definir clases, relaciones y restricciones.



Imagen tomada de: [Internet](#).

Problemas en la Adquisición del Conocimiento.

- ▶ **Conocimiento tácito:** difícil de expresar formalmente.
 - ▶ **Ejemplo:** un médico experimentado “intuye” una enfermedad por la apariencia general del paciente, pero no puede describir exactamente qué reglas utiliza mentalmente.
- ▶ **Inconsistencia o contradicción entre expertos.**
 - ▶ **Ejemplo:** dos ingenieros pueden proponer diagnósticos distintos ante la misma falla industrial, generando reglas incompatibles.
- ▶ **Costos y tiempos elevados.**
 - ▶ **Ejemplo:** entrevistar especialistas durante meses para extraer reglas de decisión puede retrasar el desarrollo del sistema.
- ▶ **Sesgos cognitivos.**
 - ▶ **Ejemplo:** un experto puede sobrevalorar casos recientes (sesgo de disponibilidad) e incorporar reglas basadas en experiencias poco representativas.

Soluciones Comunes.

- ▶ Uso de múltiples expertos para validar.
 - ▶ **Ejemplo:** comparar reglas propuestas por distintos médicos y consensuar las coincidencias.
- ▶ Técnicas de representación estructurada.
 - ▶ **Ejemplo:** emplear ontologías o reglas if-then formalizadas para evitar ambigüedades.
- ▶ Iteración incremental: adquisición y validación por ciclos.
 - ▶ **Ejemplo:** construir un prototipo inicial, probarlo en casos reales y ajustar reglas progresivamente.
- ▶ Uso de herramientas automatizadas para reducción de sesgo.
 - ▶ **Ejemplo:** combinar conocimiento experto con análisis de datos históricos mediante técnicas de minería de datos.

Ejemplo Integrado: Adquisición y Representación.

Situación: Se desea automatizar un diagnóstico clínico.

1. **Entrevista con médicos:**

Obtención de conocimiento en forma de reglas clínicas.

2. **Transcripción a lenguaje lógico:**

$\text{IF (tos} \wedge \text{fiebre)} \Rightarrow \text{gripe}$

3. **Implementación:**

Codificación de las reglas en el sistema experto.

Síntesis de Voz (Text-to-Speech, TTS)

La síntesis de voz (Text-to-Speech, TTS) es el proceso mediante el cual un sistema computacional convierte texto escrito en una señal de audio que simula el habla humana.

¿Qué hace realmente el sistema?

1. Análisis del texto

- ▶ Separación en palabras y frases.
- ▶ Interpretación de signos de puntuación.
- ▶ Normalización (ej.: “2026” → “dos mil veintiséis”).

2. Conversión a representación fonética

- ▶ Cada palabra se transforma en fonemas (unidades básicas del sonido del habla).

3. Modelado prosódico

- ▶ Entonación.
- ▶ Ritmo.
- ▶ Acento.
- ▶ Pausas.

4. Generación de la señal de audio

- ▶ Producción de la onda sonora digital que finalmente se escucha.

Biblioteca pyttsx3

- ▶ pyttsx3 es una librería de Python para **síntesis de voz** (Text-to-Speech, TTS).
- ▶ Permite convertir texto en audio sin necesidad de conexión a internet.
- ▶ Funciona utilizando los motores de voz instalados en el sistema operativo.
- ▶ Es multiplataforma: Windows, macOS y Linux.

Idea clave: No implementa el motor de voz, sino que actúa como interfaz.

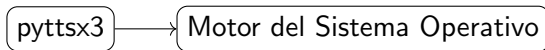
Origen de pyttsx3

- ▶ Surge como evolución de la librería pyttsx.
- ▶ pyttsx fue desarrollada por Sagun Tiwari.
- ▶ pyttsx3 es una versión compatible con Python 3.
- ▶ Proyecto de código abierto mantenido por la comunidad.

Importante: No pertenece a Google ni a una empresa privada específica.

Motores de voz utilizados

- ▶ En Windows: utiliza **SAPI5** (Microsoft Speech API).
- ▶ En macOS: utiliza **NSSpeechSynthesizer**.
- ▶ En Linux: utiliza normalmente **eSpeak**.



Funcionamiento General

El proceso de síntesis puede representarse como:



- ▶ El usuario proporciona una cadena de texto.
- ▶ La librería envía el texto al motor del sistema.
- ▶ El motor convierte texto en fonemas.
- ▶ Se genera la señal de audio.
- ▶ El sonido se reproduce o se guarda en archivo.

Ejemplo de Uso en Python

```
import pyttsx3
```

```
texto = "Este es un ejemplo de síntesis de voz."
```

```
engine = pyttsx3.init()  
engine.say(texto)  
engine.runAndWait()
```

- ▶ `init()` inicializa el motor.
- ▶ `say()` agrega texto a la cola.
- ▶ `runAndWait()` ejecuta la síntesis.

Parámetros Configurables

- ▶ Velocidad de habla:

```
engine.setProperty('rate', 150)
```

- ▶ Volumen:

```
engine.setProperty('volume', 1.0)
```

- ▶ Tipo de voz:

```
engine.setProperty('voice', voice_id)
```

Ventajas

- ▶ No requiere internet.
- ▶ Fácil instalación.
- ▶ Multiplataforma.
- ▶ Bajo consumo computacional.

Limitaciones

- ▶ Calidad dependiente del sistema operativo.
- ▶ No usa modelos neuronales avanzados.
- ▶ No permite clonación de voz.

Diagnóstico de Fallas de un Motor, por voz, usando un Sistema Experto

Objetivo General

Desarrollar un sistema que integre:

- ▶ Reconocimiento de voz.
- ▶ Procesamiento de lenguaje natural básico.
- ▶ Representación del conocimiento en lógica de primer orden.
- ▶ Inferencia automática mediante reglas.
- ▶ Síntesis de voz.

Objetivos Específicos

- ▶ Implementar reconocimiento de voz con `speech_recognition`.
- ▶ Transformar texto en hechos lógicos.
- ▶ Modelar una base de conocimiento formal.
- ▶ Implementar un motor de inferencia.
- ▶ Generar diagnósticos automáticos.
- ▶ Presentar el diagnóstico mediante síntesis de voz usando `pyttsx3`.

Contexto del Problema

Se desea diagnosticar fallas en un motor eléctrico industrial a partir de una descripción verbal.

Ejemplo de entrada por voz:

“El motor no gira y huele a quemado”

El sistema debe:

1. Convertir voz en texto.
2. Extraer síntomas.
3. Aplicar reglas lógicas.
4. Emitir un diagnóstico mediante texto y voz.

Dominio y Predicados

Constante del dominio: *motor1* **Predicados:**

- ▶ *no_gira*(*x*)
- ▶ *huele_quemado*(*x*)
- ▶ *sin_energia*(*x*)
- ▶ *vibracion_excesiva*(*x*)

Conclusiones posibles:

- ▶ *bobinado_dannado*(*x*)
- ▶ *fusible_dannado*(*x*)
- ▶ *desalineacion_mecanica*(*x*)

Base de Conocimiento Formal

Reglas en lógica de primer orden:

$$\forall x (no_gira(x) \wedge huele_quemado(x) \rightarrow bobinado_dannado(x))$$

$$\forall x (no_gira(x) \wedge sin_energia(x) \rightarrow fusible_dannado(x))$$

$$\forall x (vibracion_excesiva(x) \rightarrow desalineacion_mecanica(x))$$

Flujo del Sistema

Voz → Texto → Hechos → Reglas → Diagnóstico

1. Captura de audio.
2. Reconocimiento con Google Speech API.
3. Extracción de patrones.
4. Encadenamiento hacia adelante.
5. Síntesis de voz del diagnóstico.

Reconocimiento de Voz

```
import speech_recognition as sr

r = sr.Recognizer()
with sr.Microphone() as source:
    audio = r.listen(source)

texto = r.recognize_google(audio, language="es-ES")
```

Extracción de Hechos

Ejemplo:

- ▶ Texto reconocido:

“el motor no gira y huele a quemado”

- ▶ Hechos generados:

no_gira(motor1)

huele_quemado(motor1)

Motor de Inferencia

Si: $no_gira(motor1) \wedge huele_quemado(motor1)$

Entonces: $bobinado_dannado(motor1)$

Estrategia sugerida:

- Encadenamiento hacia adelante.

Síntesis de Voz

Archivo: *tts.py*

```
import pyttsx3
import sys

engine = pyttsx3.init()
engine.say(sys.argv[1])
engine.runAndWait()
```

Síntesis e voz:

```
import subprocess

subprocess.run(["python", "fig/Notebook/tts.py", "TEXT0"])
```

Ver código python.

Desarrollo de un Chatbot Experto con Interfaz por Voz

Diseñar e implementar un **chatbot interactivo** que:

1. Realice **reconocimiento de voz** para capturar las respuestas del usuario.
2. Procese la información mediante un **sistema experto basado en reglas** en lógica de primer orden.
3. Genere un **diagnóstico o recomendación razonada**.
4. Presente el resultado mediante **síntesis de voz**.

El **dominio** del sistema experto es **libre**, pero debe cumplir con los siguientes criterios:

- ▶ 8 reglas mínimo.
- ▶ Al menos 2 reglas encadenadas.
- ▶ 8 variables de entrada.
- ▶ 1 diagnóstico final no trivial.

Entregables

- ▶ Código funcional y comentado (en Notebook).
- ▶ Evidencia en video de máximo 10 minutos.

El código debe incluir:

- ▶ Representación formal.
- ▶ Explicación del motor de inferencia.
- ▶ Ejemplos de ejecución.

El video debe incluir:

- ▶ Explicación clara del código.
- ▶ Hacer énfasis en el Sistema Experto.
- ▶ Ejemplos de ejecución.

Algunas preguntas de análisis

1. ¿Cuál es la importancia del tema seleccionado?
2. ¿Qué limitaciones presenta el reconocimiento de voz?
3. ¿Cómo se formaliza el lenguaje natural en lógica?
4. ¿Qué ventajas ofrece la lógica simbólica?
5. ¿Cómo escalar el sistema a un entorno industrial?

Gracias por su atención.

